



UNIVERSIDAD PERUANA LOS ANDES

Facultad de Ingeniería — Escuela Profesional de Ingeniería de Sistemas y Computación



ACTIVIDAD — BASE DE DATOS II

Semana 12

Estudiante: Eduardo Fredy Ramón Puente

Código: T01311A

Curso: Base de Datos II

Docente: Ing. Raúl Fernández Bejarano

TEMA 1. Estrategias de Backup: Completo, Diferencial y Transaccional

Proyecto 1: Respaldo Completo Institucional de la Base de Datos GradosTítulos

1. Enunciado del ejercicio

La Oficina de Tecnologías de Información de la universidad requiere implementar una política de respaldo completo diario para la base de datos GradosTítulos, la cual almacena información crítica de estudiantes, trámites, evaluaciones, resoluciones y diplomas. El estudiante deberá crear un procedimiento que genere un respaldo completo de la base de datos y registre la fecha de ejecución.

2. Script de la solución en T-SQL

```
CREATE PROCEDURE sp_RespaldoCompletoGradosTítulos
AS
BEGIN
    DECLARE @Fecha VARCHAR(20) = FORMAT(GETDATE(), 'yyyyMMdd_HH:mm:ss');
    DECLARE @Ruta NVARCHAR(500) = N'C:\Backups\GradosTítulos_Full_' + @Fecha + N'.bak';

    BACKUP DATABASE GradosTítulos
    TO DISK = @Ruta
    WITH INIT, COMPRESSION,
        NAME = N'Respaldo Completo GradosTítulos',
        STATS = 10;

    PRINT 'Respaldo completo generado: ' + @Ruta
        + ' - Fecha: ' + CONVERT(VARCHAR, GETDATE(), 120);
END
GO

EXEC sp_RespaldoCompletoGradosTítulos;
```

3. Justificación técnica de la solución aplicada

BACKUP DATABASE captura una copia completa y consistente de todos los esquemas (Academico, Tramite, Evaluacion, Documento) en un solo archivo. El nombre del archivo se genera dinámicamente con marca de tiempo (yyyyMMdd_HH:mm:ss) para evitar sobrescrituras accidentales entre ejecuciones diarias. La opción COMPRESSION reduce el tamaño del archivo y el tiempo de E/S, mientras que STATS = 10 permite monitorear el progreso del respaldo en producción.

4. Buenas prácticas utilizadas en el proyecto

- Almacenar los respaldos en una unidad distinta a la de los archivos de datos (.mdf/.ldf).
- Usar una convención de nombres con fecha y hora para trazabilidad histórica.
- Habilitar COMPRESSION para optimizar espacio en disco y tiempo de ejecución.
- Automatizar la ejecución mediante un Job de SQL Server Agent programado diariamente.
- Verificar el éxito del respaldo revisando msdb.dbo.backupset.

Proyecto 2: Implementación de Backups Diferenciales para Trámites Académicos

1. Enunciado del ejercicio

La universidad necesita reducir los tiempos de respaldo debido al crecimiento de las tablas de los esquemas Tramite y Documento. Diseñe una estrategia que combine respaldos completos semanales y diferenciales diarios.

2. Script de la solución en T-SQL

```
-- Respaldo completo (ejecutar cada domingo)
BACKUP DATABASE GradosTítulos
TO DISK = N'C:\Backups\GradosTítulos_Full.bak'
WITH INIT, COMPRESSION, NAME = N'Full Semanal - GradosTítulos';
GO

-- Respaldo diferencial (ejecutar de lunes a sábado)
```

```

DECLARE @archivo NVARCHAR(500) = N'C:\Backups\GradosTitulos_Diff_'
      + REPLACE(CONVERT(VARCHAR(10), GETDATE(), 120), '-', '') + N'.bak';

DECLARE @sql NVARCHAR(1000) = N'BACKUP DATABASE GradosTitulos
TO DISK = ''' + @archivo + N'''
WITH DIFFERENTIAL, COMPRESSION, NAME = 'Diferencial Diario', STATS = 10;';

EXEC sp_executesql @sql;

```

3. Justificación técnica de la solución aplicada

Los respaldos diferenciales solo capturan las extensiones (extents) modificadas desde el último respaldo completo, lo que reduce drásticamente el tiempo y el espacio requerido frente a un respaldo completo diario, especialmente cuando las tablas Tramite y Documento crecen aceleradamente. Esta combinación equilibra el objetivo de punto de recuperación (RPO) con el uso eficiente de almacenamiento.

4. Buenas prácticas utilizadas en el proyecto

- Programar el respaldo completo en un horario de baja carga transaccional (ej. domingo 00:00).
- Monitorear el crecimiento del diferencial: si se acerca al tamaño del completo, es momento de generar un nuevo full.
- Mantener la cadena de respaldos (full + diferenciales) ordenada por fecha para la restauración.
- Automatizar ambos procesos mediante SQL Server Agent con alertas de fallo.
- Documentar la política de respaldo para el personal de TI.

Proyecto 3: Estrategia Integral Full + Differential + Log Backup

1. Enunciado del ejercicio

La base de datos GradosTitulos opera durante las 24 horas y registra continuamente solicitudes de grados y títulos. Diseñe una estrategia que contemple: respaldo completo semanal, respaldo diferencial diario y respaldo del log de transacciones cada 30 minutos.

2. Script de la solución en T-SQL

```

-- 1. Modelo de recuperación necesario para respaldos de log
ALTER DATABASE GradosTitulos SET RECOVERY FULL;
GO

-- 2. Respaldo completo semanal (domingo 00:00)
BACKUP DATABASE GradosTitulos
TO DISK = N'C:\Backups\GradosTitulos_Full.bak'
WITH INIT, COMPRESSION, NAME = N'Full Semanal';
GO

-- 3. Respaldo diferencial diario (lunes a sábado 00:00)
BACKUP DATABASE GradosTitulos
TO DISK = N'C:\Backups\GradosTitulos_Diff.bak'
WITH DIFFERENTIAL, COMPRESSION, NAME = N'Diferencial Diario';
GO

-- 4. Respaldo de log cada 30 minutos (Job de SQL Server Agent)
BACKUP LOG GradosTitulos
TO DISK = N'C:\Backups\GradosTitulos_Log.trn'
WITH COMPRESSION, NAME = N'Log cada 30 min';

```

3. Justificación técnica de la solución aplicada

El modelo de recuperación FULL es obligatorio para poder ejecutar respaldos de log y habilitar la recuperación a un punto en el tiempo. Al respaldar el log cada 30 minutos se acota la pérdida máxima de información (RPO) a 30 minutos. La combinación en capas full/diferencial/log minimiza tanto la ventana de respaldo como el tiempo total de restauración (RTO) ante un incidente.

4. Buenas prácticas utilizadas en el proyecto

- Dedicar un disco o ruta exclusiva para los archivos .trn de log.
- Monitorear el crecimiento del archivo de log para evitar llenado del disco.
- Automatizar los tres tipos de respaldo con Jobs independientes en SQL Server Agent.

- Configurar alertas por correo ante fallos en cualquiera de los Jobs.
- Probar periódicamente la secuencia completa de restauración (full+diff+log).

Proyecto 4: Diseño de Estrategia de Respaldo para Alta Disponibilidad

1. Enunciado del ejercicio

Se requiere diseñar una estrategia de recuperación ante desastres para la plataforma institucional de grados y títulos. El estudiante deberá definir respaldos completos, diferenciales y transaccionales para garantizar una recuperación máxima de 15 minutos.

2. Script de la solución en T-SQL

```
ALTER DATABASE GradosTitulos SET RECOVERY FULL;
GO

BACKUP DATABASE GradosTitulos
TO DISK = N'C:\Backups\GradosTitulos_Full.bak'
WITH INIT, COMPRESSION;
GO

BACKUP DATABASE GradosTitulos
TO DISK = N'C:\Backups\GradosTitulos_Diff.bak'
WITH DIFFERENTIAL, COMPRESSION;
GO

-- Job de Agent programado cada 15 minutos:
BACKUP LOG GradosTitulos
TO DISK = N'C:\Backups\GradosTitulos_Log.trn'
WITH COMPRESSION;
```

3. Justificación técnica de la solución aplicada

Programar el respaldo de log cada 15 minutos garantiza que, ante una falla, la pérdida máxima de información (RPO) sea de 15 minutos, cumpliendo el requerimiento institucional. El respaldo diferencial diario reduce el número de archivos de log necesarios durante la restauración, disminuyendo así el tiempo de recuperación (RTO).

4. Buenas prácticas utilizadas en el proyecto

- Definir explícitamente el RPO y RTO objetivo antes de diseñar la estrategia.
- Usar SQL Server Agent con reintentos automáticos ante fallos de Job.
- Evaluar el uso de Always On Availability Groups como complemento a los respaldos.
- Medir periódicamente el tiempo real de restauración contra el RTO definido.
- Mantener los tres tipos de respaldo en ubicaciones con redundancia.

Proyecto 5: Implementación de una Estrategia Corporativa de Respaldo para GradosTitulos

1. Enunciado del ejercicio

La Universidad ha identificado que la base de datos GradosTitulos es crítica para la gestión de expedientes de grados y títulos. Debido al incremento de trámites registrados en los esquemas Academico, Tramite, Evaluacion y Documento, se requiere implementar una estrategia avanzada de respaldo que garantice una pérdida máxima de información de 15 minutos. Diseñe una solución que incluya: respaldo completo cada domingo, respaldo diferencial diario, respaldo del log de transacciones cada 15 minutos, compresión de respaldos y verificación automática de integridad.

2. Script de la solución en T-SQL

```
ALTER DATABASE GradosTitulos SET RECOVERY FULL;
GO

-- Respaldo completo (domingo)
BACKUP DATABASE GradosTitulos
TO DISK = N'C:\Backups\GradosTitulos_Full.bak'
WITH INIT, COMPRESSION, CHECKSUM, NAME = N'Full Dominical';
```

```
GO

-- Respaldo diferencial (diario)
BACKUP DATABASE GradosTitulos
TO DISK = N'C:\Backups\GradosTitulos_Diff.bak'
WITH DIFFERENTIAL, COMPRESSION, CHECKSUM, NAME = N'Diferencial Diario';
GO

-- Respaldo de log (cada 15 minutos)
BACKUP LOG GradosTitulos
TO DISK = N'C:\Backups\GradosTitulos_Log.trn'
WITH COMPRESSION, CHECKSUM, NAME = N'Log 15min';
GO

-- Verificación automática de integridad
RESTORE VERIFYONLY FROM DISK = N'C:\Backups\GradosTitulos_Full.bak';
```

3. Justificación técnica de la solución aplicada

La opción CHECKSUM detecta corrupción de páginas durante el proceso de respaldo, mientras que RESTORE VERIFYONLY confirma que el conjunto de respaldo es legible y restaurable sin necesidad de ejecutar una restauración completa. La compresión reduce la carga de E/S y el espacio de almacenamiento requerido para los cuatro esquemas críticos.

4. Buenas prácticas utilizadas en el proyecto

- Verificar cada respaldo generado con RESTORE VERIFYONLY inmediatamente después de crearlo.
- Habilitar siempre CHECKSUM para detectar corrupción temprana.
- Mantener copias en al menos dos ubicaciones (local y offsite).
- Documentar formalmente el RPO/RTO acordado con la Oficina de TI.
- Automatizar la verificación como parte del mismo Job de respaldo.

Proyecto 6: Diseño de Estrategia de Respaldo para Recuperación ante Desastres

1. Enunciado del ejercicio

La Oficina de Tecnologías de Información requiere implementar una estrategia de Disaster Recovery para la base de datos GradosTitulos. Diseñe un esquema de respaldo que permita reconstruir completamente la base de datos en un servidor alternativo ante la pérdida total del servidor principal.

2. Script de la solución en T-SQL

```
-- En el servidor primario: generar full + diferencial + log hacia el sitio DR
BACKUP DATABASE GradosTitulos
TO DISK = N'\\ServidorDR\Backups\GradosTitulos_Full.bak'
WITH INIT, COMPRESSION;
GO

BACKUP DATABASE GradosTitulos
TO DISK = N'\\ServidorDR\Backups\GradosTitulos_Diff.bak'
WITH DIFFERENTIAL, COMPRESSION;
GO

BACKUP LOG GradosTitulos
TO DISK = N'\\ServidorDR\Backups\GradosTitulos_Log.trn'
WITH COMPRESSION;
GO

-- En el servidor DR: reconstrucción completa
RESTORE DATABASE GradosTitulos
FROM DISK = N'\\ServidorDR\Backups\GradosTitulos_Full.bak'
WITH NORECOVERY, REPLACE;
GO

RESTORE DATABASE GradosTitulos
FROM DISK = N'\\ServidorDR\Backups\GradosTitulos_Diff.bak'
WITH NORECOVERY;
GO
```

```
RESTORE LOG GradosTitulos
FROM DISK = N'\\ServidorDR\Backups\GradosTitulos_Log.trn'
WITH RECOVERY;
```

3. Justificación técnica de la solución aplicada

Escribir los respaldos directamente en una ruta de red hacia el sitio de contingencia (o replicarlos posteriormente) permite reconstruir la base de datos sin depender del servidor principal averiado. La cláusula NORECOVERY mantiene la base en estado de restauración para poder encadenar los siguientes respaldos; la RECOVERY final pone la base en línea.

4. Buenas prácticas utilizadas en el proyecto

- Replicar o almacenar los respaldos en un sitio físicamente distinto al servidor principal.
 - Realizar simulacros periódicos de failover hacia el servidor DR.
 - Documentar el procedimiento de recuperación en un runbook accesible al equipo de TI.
 - Usar una cuenta de servicio con privilegios mínimos para el acceso a la carpeta compartida.
 - Automatizar la copia de archivos con herramientas como robocopy con validación de integridad.
-

TEMA 2. Restauración de Bases de Datos en Distintos Escenarios

Proyecto 1: Configuración segura de servicios SQL Server

1. Enunciado del ejercicio

Un administrador eliminó accidentalmente registros relacionados con expedientes de titulación. Restaure la base de datos utilizando el respaldo más reciente.

2. Script de la solución en T-SQL

```
USE master;
ALTER DATABASE GradosTitulos SET SINGLE_USER WITH ROLLBACK IMMEDIATE;

RESTORE DATABASE GradosTitulos
FROM DISK = N'C:\Backups\GradosTitulos_Full.bak'
WITH REPLACE, RECOVERY;

ALTER DATABASE GradosTitulos SET MULTI_USER;
```

3. Justificación técnica de la solución aplicada

SINGLE_USER WITH ROLLBACK IMMEDIATE fuerza la desconexión de sesiones activas para poder restaurar en modo exclusivo. REPLACE permite sobrescribir la base actual (con datos eliminados por error), y RECOVERY deja la base lista para usarse, dado que este es el respaldo más reciente y completo disponible.

4. Buenas prácticas utilizadas en el proyecto

- Verificar la validez del respaldo con RESTORE VERIFYONLY antes de restaurar.
- Notificar a los usuarios antes de forzar la desconexión de sesiones.
- Confirmar que el archivo de respaldo corresponde a la fecha correcta con RESTORE HEADERONLY.
- Ejecutar DBCC CHECKDB después de la restauración para validar consistencia.
- Restringir el permiso de ejecutar restauraciones destructivas solo a roles de DBA.

Proyecto 2: Restauración en un Servidor Alternativo

1. Enunciado del ejercicio

La universidad requiere habilitar un servidor de contingencia para pruebas y recuperación. Restaure la base de datos GradosTitulos en una nueva instancia SQL Server.

2. Script de la solución en T-SQL

```
RESTORE DATABASE GradosTitulos
FROM DISK = N'\\Servidor01\Backups\GradosTitulos_Full.bak'
WITH MOVE 'GradosTitulos' TO 'D:\SQLData\GradosTitulos.mdf',
      MOVE 'GradosTitulos_log' TO 'D:\SQLLogs\GradosTitulos_log.ldf',
      REPLACE, RECOVERY;
```

3. Justificación técnica de la solución aplicada

La cláusula MOVE reubica los archivos lógicos de datos y log a rutas existentes en el nuevo servidor, ya que la estructura de directorios puede diferir del servidor origen. REPLACE permite crear la base incluso si existiera una base con distinto linaje en el destino.

4. Buenas prácticas utilizadas en el proyecto

- Consultar previamente los nombres lógicos de archivo con RESTORE FILELISTONLY.
- Verificar espacio disponible en la unidad de destino antes de restaurar.
- Validar compatibilidad de versión/edición de SQL Server entre origen y destino.
- Probar la conectividad de las aplicaciones tras la restauración.
- Documentar el propósito del servidor (contingencia/pruebas) para evitar confusión operativa.

Proyecto 3: Recuperación después de Corrupción del Archivo MDF

1. Enunciado del ejercicio

El archivo principal de datos presenta corrupción física. Implemente el procedimiento de restauración completo utilizando los respaldos disponibles.

2. Script de la solución en T-SQL

```
ALTER DATABASE GradosTitulos SET OFFLINE WITH ROLLBACK IMMEDIATE;

RESTORE DATABASE GradosTitulos
FROM DISK = N'C:\Backups\GradosTitulos_Full.bak'
WITH REPLACE, NORECOVERY;

RESTORE DATABASE GradosTitulos
FROM DISK = N'C:\Backups\GradosTitulos_Diff.bak'
WITH NORECOVERY;

RESTORE LOG GradosTitulos
FROM DISK = N'C:\Backups\GradosTitulos_Log.trn'
WITH RECOVERY;
```

3. Justificación técnica de la solución aplicada

Al estar corrupto el archivo MDF, la base debe ponerse OFFLINE antes de restaurar. La secuencia full + diferencial + log reconstruye la base hasta el punto más reciente y consistente disponible; NORECOVERY en los pasos intermedios permite continuar encadenando restauraciones, mientras que la RECOVERY final completa el proceso.

4. Buenas prácticas utilizadas en el proyecto

- Ejecutar DBCC CHECKDB inmediatamente después de recuperar la base.
- Investigar la causa raíz de la corrupción (hardware de almacenamiento, RAID, disco).
- Conservar el archivo corrupto para análisis forense si el incidente lo amerita.
- Probar periódicamente este escenario de restauración en un ambiente de pruebas.

Proyecto 4: Recuperación ante Desastre Total del Servidor

1. Enunciado del ejercicio

Una falla de almacenamiento destruyó completamente el servidor donde se encontraba la base GradosTitulos. Diseñe el proceso de recuperación completa.

2. Script de la solución en T-SQL

```
-- En el nuevo servidor:
RESTORE DATABASE GradosTitulos
FROM DISK = N'\\BackupServer\Backups\GradosTitulos_Full.bak'
WITH MOVE 'GradosTitulos' TO 'E:\SQLData\GradosTitulos.mdf',
      MOVE 'GradosTitulos_log' TO 'E:\SQLLogs\GradosTitulos_log.ldf',
      NORECOVERY, REPLACE;

RESTORE DATABASE GradosTitulos
FROM DISK = N'\\BackupServer\Backups\GradosTitulos_Diff.bak'
WITH NORECOVERY;

RESTORE LOG GradosTitulos
FROM DISK = N'\\BackupServer\Backups\GradosTitulos_Log.trn'
WITH RECOVERY;
```

3. Justificación técnica de la solución aplicada

Al no existir ya el servidor original, los respaldos deben residir en almacenamiento externo o fuera de sitio. La instalación de un nuevo servidor SQL Server junto con la secuencia completa full+diferencial+log reconstruye la base de datos desde cero.

4. Buenas prácticas utilizadas en el proyecto

- Mantener almacenamiento de respaldos geo-redundante (offsite o en la nube).
- Contar con un runbook documentado de recuperación ante desastres (DRP).
- Realizar simulacros periódicos de DR para validar tiempos reales de recuperación.

- Utilizar infraestructura como código para aprovisionar servidores rápidamente.

Proyecto 5: Recuperación Completa después de Corrupción del Archivo MDF

1. Enunciado del ejercicio

Se detectó corrupción física en el archivo principal de datos de la base GradosTitulos, impidiendo el acceso a las tablas relacionadas con expedientes de titulación. Recupere completamente la base utilizando la secuencia de respaldos disponibles.

2. Script de la solución en T-SQL

```
ALTER DATABASE GradosTitulos SET OFFLINE WITH ROLLBACK IMMEDIATE;

RESTORE DATABASE GradosTitulos
FROM DISK = N'C:\Backups\GradosTitulos_Full.bak'
WITH REPLACE, NORECOVERY;

RESTORE DATABASE GradosTitulos
FROM DISK = N'C:\Backups\GradosTitulos_Diff.bak'
WITH NORECOVERY;

RESTORE LOG GradosTitulos
FROM DISK = N'C:\Backups\GradosTitulos_Log.trn'
WITH RECOVERY;

DBCC CHECKDB('GradosTitulos') WITH NO_INFOMSGS;
SELECT COUNT(*) AS TotalTramites FROM Tramite.Tramite;
```

3. Justificación técnica de la solución aplicada

La secuencia full+diferencial+log restaura la base al último estado consistente posible. La verificación posterior con DBCC CHECKDB y un conteo de registros del esquema Tramite confirma que las tablas afectadas por la corrupción quedaron correctamente recuperadas.

4. Buenas prácticas utilizadas en el proyecto

- Validar específicamente las tablas del esquema afectado tras la restauración.
- Ejecutar DBCC CHECKDB antes de habilitar el acceso a usuarios finales.
- Registrar el incidente y el tiempo de indisponibilidad para reportes de TI.
- Revisar el estado del disco/controladora para prevenir corrupciones futuras.

Proyecto 6: Restauración de la Base de Datos en un Servidor de Contingencia

1. Enunciado del ejercicio

La Universidad requiere habilitar un centro alternativo de recuperación para continuar los procesos de grados y títulos. Restaure la base en una instancia secundaria.

2. Script de la solución en T-SQL

```
RESTORE DATABASE GradosTitulos
FROM DISK = N'\\ServidorContingencia\Backups\GradosTitulos_Full.bak'
WITH MOVE 'GradosTitulos' TO 'F:\SQLData\GradosTitulos.mdf',
      MOVE 'GradosTitulos_log' TO 'F:\SQLLogs\GradosTitulos_log.ldf',
      NORECOVERY, REPLACE;

RESTORE LOG GradosTitulos
FROM DISK = N'\\ServidorContingencia\Backups\GradosTitulos_Log.trn'
WITH RECOVERY;
```

3. Justificación técnica de la solución aplicada

Dejar la base en NORECOVERY tras el respaldo completo permite continuar aplicando respaldos de log posteriores, lo que resulta útil si se implementa una solución de log shipping para mantener sincronizado el servidor de contingencia. La RECOVERY final habilita la base para uso operativo.

4. Buenas prácticas utilizadas en el proyecto

- Evaluar log shipping o Always On para mantener el servidor de contingencia sincronizado en forma continua.
 - Validar permisos y conectividad de red hacia el servidor secundario.
 - Programar pruebas periódicas de restauración en el servidor de contingencia.
 - Documentar el procedimiento de failover y failback.
-

TEMA 3. Mantenimiento y Verificación de Copias de Seguridad

Proyecto 1: Verificación de Integridad de Backups

1. Enunciado del ejercicio

Verifique que todos los respaldos generados para GradosTítulos sean recuperables.

2. Script de la solución en T-SQL

```
RESTORE VERIFYONLY FROM DISK = N'C:\Backups\GradosTítulos_Full.bak';
RESTORE VERIFYONLY FROM DISK = N'C:\Backups\GradosTítulos_Diff.bak';
RESTORE VERIFYONLY FROM DISK = N'C:\Backups\GradosTítulos_Log.trn';
```

3. Justificación técnica de la solución aplicada

RESTORE VERIFYONLY valida la estructura del conjunto de respaldo (encabezados, checksums si existen) sin ejecutar una restauración real, confirmando que el archivo no está corrupto ni truncado antes de necesitarlo en un incidente real.

4. Buenas prácticas utilizadas en el proyecto

- Ejecutar VERIFYONLY inmediatamente después de cada Job de respaldo.
- Combinar con la opción CHECKSUM durante el respaldo para mayor confiabilidad.
- Registrar los resultados de verificación en una tabla de auditoría.
- Complementar con restauraciones de prueba completas de forma periódica, ya que VERIFYONLY no garantiza el 100% de las páginas.

Proyecto 2: Inventario Histórico de Copias de Seguridad

1. Enunciado del ejercicio

Construya un reporte que muestre todos los respaldos realizados durante el último mes.

2. Script de la solución en T-SQL

```
SELECT
    bs.database_name AS BaseDatos,
    bs.backup_start_date AS Inicio,
    bs.backup_finish_date AS Fin,
    CASE bs.type
        WHEN 'D' THEN 'Completo'
        WHEN 'I' THEN 'Diferencial'
        WHEN 'L' THEN 'Log'
    END AS TipoRespaldo,
    bmf.physical_device_name AS Ubicacion,
    bs.backup_size / 1024 / 1024 AS TamañoMB
FROM msdb.dbo.backupset bs
INNER JOIN msdb.dbo.backupmediafamily bmf
    ON bs.media_set_id = bmf.media_set_id
WHERE bs.database_name = 'GradosTítulos'
    AND bs.backup_start_date >= DATEADD(MONTH, -1, GETDATE())
ORDER BY bs.backup_start_date DESC;
```

3. Justificación técnica de la solución aplicada

Las tablas del sistema msdb.dbo.backupset y backupmediafamily almacenan automáticamente el historial de respaldos ejecutados por SQL Server. Consultarlas evita depender de herramientas externas para auditar la política de respaldo institucional.

4. Buenas prácticas utilizadas en el proyecto

- Programar esta consulta como reporte automático (SSRS o Job con envío de correo).
- Depurar periódicamente el historial antiguo con sp_delete_backuphistory.
- Cruzar el resultado con los archivos físicos existentes en disco para detectar inconsistencias.
- Restringir el acceso a msdb solo a personal DBA autorizado.

Proyecto 3: Detección de Copias de Seguridad Obsoletas

1. Enunciado del ejercicio

Identifique copias de seguridad que excedan la política institucional de retención.

2. Script de la solución en T-SQL

```
SELECT
    bs.database_name AS BaseDatos,
    bmf.physical_device_name AS Archivo,
    bs.backup_start_date AS FechaRespaldo,
    DATEDIFF(DAY, bs.backup_start_date, GETDATE()) AS DiasAntigüedad
FROM msdb.dbo.backupset bs
INNER JOIN msdb.dbo.backupmediafamily bmf
    ON bs.media_set_id = bmf.media_set_id
WHERE bs.database_name = 'GradosTitulos'
    AND DATEDIFF(DAY, bs.backup_start_date, GETDATE()) > 90
ORDER BY bs.backup_start_date;
```

3. Justificación técnica de la solución aplicada

Se compara la fecha de cada respaldo contra el umbral de retención (90 días) definido por la política institucional, identificando los archivos que ya superaron ese límite y son candidatos a archivado o eliminación.

4. Buenas prácticas utilizadas en el proyecto

- Automatizar esta consulta como Job programado con reporte de salida.
- Revisar manualmente el listado antes de eliminar los archivos físicos.
- Conservar al menos una copia de largo plazo por excepción de cumplimiento normativo.
- Registrar en un log toda eliminación de respaldos para fines de auditoría.

Proyecto 4: Sistema Automatizado de Verificación

1. Enunciado del ejercicio

Diseñe una solución que valide diariamente la integridad de todos los respaldos de GradosTitulos.

2. Script de la solución en T-SQL

```
CREATE PROCEDURE sp_VerificarIntegridadBackups
AS
BEGIN
    DECLARE @archivo NVARCHAR(500);

    DECLARE cur CURSOR FOR
        SELECT DISTINCT bmf.physical_device_name
        FROM msdb.dbo.backupset bs
        INNER JOIN msdb.dbo.backupmediafamily bmf ON bs.media_set_id = bmf.media_set_id
        WHERE bs.database_name = 'GradosTitulos'
            AND bs.backup_start_date >= DATEADD(DAY, -1, GETDATE());

    OPEN cur;
    FETCH NEXT FROM cur INTO @archivo;

    WHILE @@FETCH_STATUS = 0
    BEGIN
        BEGIN TRY
            RESTORE VERIFYONLY FROM DISK = @archivo;
            PRINT 'OK: ' + @archivo;
        END TRY
        BEGIN CATCH
            PRINT 'ERROR en: ' + @archivo + ' - ' + ERROR_MESSAGE();
        END CATCH
        FETCH NEXT FROM cur INTO @archivo;
    END

    CLOSE cur;
    DEALLOCATE cur;
END
GO
```

```
-- Programar ejecución diaria mediante SQL Server Agent
```

3. Justificación técnica de la solución aplicada

El cursor recorre dinámicamente todos los respaldos registrados en las últimas 24 horas; el bloque TRY/CATCH captura errores de verificación sin detener el ciclo, logrando una auditoría diaria completa y automática.

4. Buenas prácticas utilizadas en el proyecto

- Registrar los resultados en una tabla en lugar de solo PRINT, para trazabilidad persistente.
- Enviar alertas por correo (sp_send_dbmail) cuando se detecte un ERROR.
- Programar la ejecución en horario de baja carga mediante SQL Server Agent.
- Mantener el procedimiento idempotente para permitir reejecuciones seguras.

Proyecto 5: Sistema Automatizado de Validación de Backups

1. Enunciado del ejercicio

Diseñe un mecanismo que valide diariamente todos los respaldos de la base GradosTítulos.

2. Screenshot de la solución en T-SQL

```
CREATE TABLE LogValidacionBackups (
    Id INT IDENTITY PRIMARY KEY,
    Archivo NVARCHAR(500),
    Resultado VARCHAR(20),
    Mensaje NVARCHAR(500),
    FechaValidacion DATETIME DEFAULT GETDATE()
);
GO

CREATE PROCEDURE sp_ValidarBackupsDiario
AS
BEGIN
    DECLARE @archivo NVARCHAR(500);
    DECLARE cur CURSOR FOR
        SELECT DISTINCT bmf.physical_device_name
        FROM msdb.dbo.backupset bs
        INNER JOIN msdb.dbo.backupmediafamily bmf ON bs.media_set_id = bmf.media_set_id
        WHERE bs.database_name = 'GradosTítulos'
        AND bs.backup_start_date >= DATEADD(DAY, -1, GETDATE());

    OPEN cur;
    FETCH NEXT FROM cur INTO @archivo;
    WHILE @@FETCH_STATUS = 0
    BEGIN
        BEGIN TRY
            RESTORE VERIFYONLY FROM DISK = @archivo;
            INSERT INTO LogValidacionBackups(Archivo, Resultado, Mensaje)
            VALUES (@archivo, 'OK', 'Verificado correctamente');
        END TRY
        BEGIN CATCH
            INSERT INTO LogValidacionBackups(Archivo, Resultado, Mensaje)
            VALUES (@archivo, 'ERROR', ERROR_MESSAGE());
        END CATCH
        FETCH NEXT FROM cur INTO @archivo;
    END
    CLOSE cur; DEALLOCATE cur;
END
```

3. Justificación técnica de la solución aplicada

Persistir los resultados en una tabla de auditoría (LogValidacionBackups) permite un seguimiento histórico y generación de reportes de cumplimiento, a diferencia de una salida transitoria por PRINT que se pierde al cerrar la sesión.

4. Buenas prácticas utilizadas en el proyecto

- Definir una política de retención/índice para la tabla de log.

- Configurar alertas automáticas ante la inserción de registros con Resultado = 'ERROR'.
- Restringir permisos de escritura sobre la tabla de auditoría.
- Ejecutar el Job fuera del horario laboral para no afectar el rendimiento.

Proyecto 6: Auditoría Integral del Historial de Copias de Seguridad

1. Enunciado del ejercicio

La Dirección de TI solicita un reporte detallado de los respaldos ejecutados sobre GradosTítulos durante los últimos 90 días.

2. Script de la solución en T-SQL

```
SELECT
    bs.database_name AS BaseDatos,
    CASE bs.type WHEN 'D' THEN 'Completo' WHEN 'I' THEN 'Diferencial' WHEN 'L' THEN 'Log' END AS Tipo,
    bs.backup_start_date AS Inicio,
    bs.backup_finish_date AS Fin,
    DATEDIFF(SECOND, bs.backup_start_date, bs.backup_finish_date) AS DuracionSeg,
    bs.backup_size / 1024 / 1024 AS TamanoMB,
    bmf.physical_device_name AS Ubicacion,
    bs.user_name AS EjecutadoPor
FROM msdb.dbo.backupset bs
INNER JOIN msdb.dbo.backupmediafamily bmf ON bs.media_set_id = bmf.media_set_id
WHERE bs.database_name = 'GradosTítulos'
    AND bs.backup_start_date >= DATEADD(DAY, -90, GETDATE())
ORDER BY bs.backup_start_date DESC;
```

3. Justificación técnica de la solución aplicada

La consulta integra tipo de respaldo, duración, tamaño, ubicación física y usuario ejecutor en un solo reporte, satisfaciendo el requerimiento de auditoría formal solicitado por la Dirección de TI.

4. Buenas prácticas utilizadas en el proyecto

- Exportar el resultado a Excel/PDF para su entrega formal.
- Programar la generación automática del reporte mensual.
- Restringir el acceso a msdb solo a DBAs autorizados.
- Conservar los reportes de auditoría según la política institucional de retención documental.

TEMA 4. Uso de BACKUP DATABASE y RESTORE DATABASE

Proyecto 1: Respaldo Manual de la Base GradosTítulos

1. Enunciado del ejercicio

Realice un respaldo completo utilizando únicamente instrucciones T-SQL.

2. Script de la solución en T-SQL

```
BACKUP DATABASE GradosTítulos
TO DISK = N'C:\Backups\GradosTítulos_Manual.bak'
WITH INIT, COMPRESSION, NAME = N'Respaldo Manual GradosTítulos', STATS = 10;
```

3. Justificación técnica de la solución aplicada

BACKUP DATABASE en su forma más simple satisface el requerimiento de utilizar únicamente instrucciones T-SQL. INIT garantiza un conjunto de respaldo limpio y COMPRESSION optimiza el almacenamiento requerido.

4. Buenas prácticas utilizadas en el proyecto

- Asignar un nombre descriptivo al respaldo mediante la opción NAME.
- Verificar el mensaje de éxito y el progreso reportado por STATS.
- Almacenar el respaldo en una unidad dedicada distinta a los datos.
- Ejecutar RESTORE VERIFYONLY tras finalizar el respaldo.

Proyecto 2: Restauración Completa desde Archivo BAK

1. Enunciado del ejercicio

Recupere la base de datos utilizando un archivo de respaldo existente.

2. Script de la solución en T-SQL

```
USE master;
ALTER DATABASE GradosTítulos SET SINGLE_USER WITH ROLLBACK IMMEDIATE;

RESTORE DATABASE GradosTítulos
FROM DISK = N'C:\Backups\GradosTítulos_Manual.bak'
WITH REPLACE, RECOVERY, STATS = 10;

ALTER DATABASE GradosTítulos SET MULTI_USER;
```

3. Justificación técnica de la solución aplicada

Colocar la base en SINGLE_USER permite obtener acceso exclusivo requerido por RESTORE; REPLACE autoriza sobrescribir la base existente con el contenido del archivo .bak, y RECOVERY la deja operativa de inmediato al ser el único archivo de respaldo aplicado.

4. Buenas prácticas utilizadas en el proyecto

- Confirmar el archivo de respaldo correcto con RESTORE HEADERONLY antes de restaurar.
- Informar a los usuarios sobre la indisponibilidad temporal del sistema.
- Regresar la base a MULTI_USER solo después de validar la restauración.
- Ejecutar DBCC CHECKDB posterior a la restauración.

Proyecto 3: Automatización de Backups y Restores

1. Enunciado del ejercicio

Diseñe procedimientos almacenados que automaticen respaldos y restauraciones.

2. Script de la solución en T-SQL

```
CREATE PROCEDURE sp_BackupGradosTítulos
AS
BEGIN
    DECLARE @archivo NVARCHAR(500) = N'C:\Backups\GradosTítulos_'
```

```

        + FORMAT(GETDATE(), 'yyyyMMdd_HH:mm:ss') + N'.bak';
BACKUP DATABASE GradosTitulos TO DISK = @archivo
WITH COMPRESSION, INIT, STATS = 10;
END
GO

CREATE PROCEDURE sp_RestoreGradosTitulos
    @ArchivoBak NVARCHAR(500)
AS
BEGIN
    ALTER DATABASE GradosTitulos SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    RESTORE DATABASE GradosTitulos FROM DISK = @ArchivoBak
    WITH REPLACE, RECOVERY;
    ALTER DATABASE GradosTitulos SET MULTI_USER;
END

```

3. Justificación técnica de la solución aplicada

Encapsular la lógica de respaldo y restauración en procedimientos parametrizados estandariza las operaciones y reduce el error humano; el nombre de archivo dinámico evita sobrescrituras, y el parámetro @ArchivoBak permite restaurar cualquier archivo seleccionado.

4. Buenas prácticas utilizadas en el proyecto

- Otorgar permisos de EXECUTE únicamente al rol de DBA.
- Registrar cada ejecución en una tabla de bitácora.
- Validar la existencia del archivo con xp_fileexist antes de restaurar.
- Programar sp_BackupGradosTitulos mediante SQL Server Agent.

Proyecto 4: Framework Institucional de Recuperación

1. Enunciado del ejercicio

Construya un conjunto de scripts reutilizables para respaldo y restauración de todas las bases institucionales.

2. Script de la solución en T-SQL

```

CREATE PROCEDURE sp_BackupCompletoGenerico
    @NombreBD SYSNAME
AS
BEGIN
    DECLARE @archivo NVARCHAR(500) = N'C:\Backups\' + @NombreBD + '_'
        + FORMAT(GETDATE(), 'yyyyMMdd_HH:mm:ss') + N'.bak';
    DECLARE @sql NVARCHAR(1000) = N'BACKUP DATABASE [' + @NombreBD
        + N'] TO DISK = N''' + @archivo + N''' WITH COMPRESSION, INIT, STATS = 10;';
    EXEC sp_executesql @sql;
END
GO

CREATE PROCEDURE sp_RestoreGenerico
    @NombreBD SYSNAME,
    @ArchivoBak NVARCHAR(500)
AS
BEGIN
    DECLARE @sql NVARCHAR(1000);
    SET @sql = N'ALTER DATABASE [' + @NombreBD + N'] SET SINGLE_USER WITH ROLLBACK IMMEDIATE;';
    EXEC sp_executesql @sql;

    SET @sql = N'RESTORE DATABASE [' + @NombreBD + N'] FROM DISK = N''' + @ArchivoBak
        + N''' WITH REPLACE, RECOVERY;';
    EXEC sp_executesql @sql;

    SET @sql = N'ALTER DATABASE [' + @NombreBD + N'] SET MULTI_USER;';
    EXEC sp_executesql @sql;
END

```

3. Justificación técnica de la solución aplicada

Parametrizar el nombre de la base de datos mediante SQL dinámico (sp_executesql) hace que el framework sea reutilizable para todas las bases institucionales y no solo para GradosTítulos.

4. Buenas prácticas utilizadas en el proyecto

- Validar @NombreBD contra sys.databases para prevenir inyección SQL.
- Centralizar estos procedimientos en una base de datos administrativa dedicada.
- Restringir los permisos de ejecución a personal DBA.
- Versionar los scripts en un repositorio de control de versiones.

Proyecto 5: Framework Automatizado de Respaldo Institucional

1. Enunciado del ejercicio

Desarrolle un procedimiento almacenado reutilizable que genere respaldos completos de GradosTítulos utilizando fecha y hora en el nombre del archivo.

2. Script de la solución en T-SQL

```
CREATE PROCEDURE sp_RespaldoAutomatizadoGradosTítulos
AS
BEGIN
    DECLARE @archivo NVARCHAR(500) = N'C:\Backups\GradosTítulos_'
        + FORMAT(GETDATE(), 'yyyyMMdd_HH:mm:ss') + N'.bak';

    BACKUP DATABASE GradosTítulos
    TO DISK = @archivo
    WITH COMPRESSION, INIT, CHECKSUM, NAME = N'Respaldo Automatizado', STATS = 10;

    PRINT 'Respaldo generado en: ' + @archivo;
END
```

3. Justificación técnica de la solución aplicada

El nombramiento con marca de tiempo (yyyyMMdd_HH:mm:ss) evita sobrescrituras entre ejecuciones y facilita la posterior aplicación de políticas de retención (Tema 3 y 5) sobre archivos identificables por fecha.

4. Buenas prácticas utilizadas en el proyecto

- Habilitar CHECKSUM para detección temprana de corrupción.
- Registrar cada ejecución mediante PRINT o una tabla de auditoría.
- Programar el procedimiento diariamente vía SQL Server Agent.
- Aplicar posteriormente una política de retención sobre los archivos generados.

Proyecto 6: Plataforma de Restauración Automatizada

1. Enunciado del ejercicio

Construya un procedimiento que permita restaurar automáticamente GradosTítulos desde cualquier archivo BAK seleccionado.

2. Script de la solución en T-SQL

```
CREATE PROCEDURE sp_RestaurarDesdeArchivo
    @ArchivoBak NVARCHAR(500)
AS
BEGIN
    IF NOT EXISTS (SELECT 1 FROM sys.databases WHERE name = 'GradosTítulos')
    BEGIN
        RAISERROR('La base de datos GradosTítulos no existe.', 16, 1);
        RETURN;
    END

    ALTER DATABASE GradosTítulos SET SINGLE_USER WITH ROLLBACK IMMEDIATE;

    RESTORE DATABASE GradosTítulos
    FROM DISK = @ArchivoBak
    WITH REPLACE, RECOVERY, STATS = 10;
```

```
ALTER DATABASE GradosTítulos SET MULTI_USER;  
  
PRINT 'Restauración completada desde: ' + @ArchivoBak;  
END
```

3. Justificación técnica de la solución aplicada

El parámetro @ArchivoBak generaliza la operación de restauración para cualquier archivo de respaldo disponible, otorgando flexibilidad operativa; la validación previa evita errores al intentar restaurar sobre una base inexistente.

4. Buenas prácticas utilizadas en el proyecto

- Validar la existencia física del archivo con xp_fileexist antes de restaurar.
 - Registrar cada restauración ejecutada, incluyendo el archivo utilizado.
 - Restringir el permiso de EXECUTE únicamente a roles de DBA.
 - Probar el procedimiento en un ambiente de no producción antes de su uso real.
-

TEMA 5. Planes de Mantenimiento y Políticas de Retención

Proyecto 1: Creación de un Plan de Mantenimiento Semanal

1. Enunciado del ejercicio

Creación de un Plan de Mantenimiento Semanal para la base de datos GradosTítulos.

2. Script de la solución en T-SQL

```
USE msdb;

EXEC sp_add_job @job_name = N'Plan_Mantenimiento_Semanal_GradosTítulos';

EXEC sp_add_jobstep
    @job_name = N'Plan_Mantenimiento_Semanal_GradosTítulos',
    @step_name = N'Backup Completo',
    @subsystem = N'TSQL',
    @command = N'BACKUP DATABASE GradosTítulos TO DISK = ''C:\Backups\GradosTítulos_Full.bak'' WITH COMPRESSI
ON, INIT;',
    @database_name = N'GradosTítulos';

EXEC sp_add_schedule
    @schedule_name = N'Semanal_Domingo',
    @freq_type = 8,
    @freq_interval = 1,
    @active_start_time = 000000;

EXEC sp_attach_schedule
    @job_name = N'Plan_Mantenimiento_Semanal_GradosTítulos',
    @schedule_name = N'Semanal_Domingo';

EXEC sp_add_jobserver
    @job_name = N'Plan_Mantenimiento_Semanal_GradosTítulos';
```

3. Justificación técnica de la solución aplicada

Los procedimientos del sistema en msdb permiten crear Jobs de SQL Server Agent completamente vía T-SQL, equivalente al Asistente de Planes de Mantenimiento (GUI), pero con la ventaja de ser reproducible y versionable como script.

4. Buenas prácticas utilizadas en el proyecto

- Nombrar Jobs y programaciones de forma descriptiva.
- Asignar el Job a una cuenta de servicio con privilegios adecuados.
- Configurar notificaciones por correo ante fallos del Job.
- Conservar el historial de ejecución del Job para auditoría.

Proyecto 2: Política de Retención de 90 Días

1. Enunciado del ejercicio

Implemente una política institucional para conservar respaldos durante 90 días.

2. Script de la solución en T-SQL

```
CREATE PROCEDURE sp_AplicarPoliticaRetencion90
AS
BEGIN
    DECLARE @FechaLimite DATETIME = DATEADD(DAY, -90, GETDATE());

    -- Elimina el historial en msdb de respaldos más antiguos que 90 días
    EXEC msdb.dbo.sp_delete_backuphistory @oldest_date = @FechaLimite;

    -- Elimina los archivos físicos obsoletos (requiere xp_cmdshell habilitado)
    DECLARE @cmd NVARCHAR(500) =
        N'forfiles /p "C:\Backups" /d -90 /m *.bak /c "cmd /c del @path"';
    EXEC xp_cmdshell @cmd;
END
```

3. Justificación técnica de la solución aplicada

sp_delete_backuphistory depura los metadatos en msdb más allá de la ventana de retención definida; el comando forfiles ejecutado mediante xp_cmdshell elimina los archivos físicos correspondientes, aplicando así la política de 90 días de manera automatizada.

4. Buenas prácticas utilizadas en el proyecto

- Habilitar xp_cmdshell únicamente de forma temporal y con permisos restringidos por el riesgo de seguridad que implica.
- Probar manualmente el comando forfiles antes de automatizarlo.
- Conservar al menos un respaldo de archivo fuera de la rotación para cumplimiento normativo.
- Registrar en un log cada eliminación realizada.

Proyecto 3: Plan de Mantenimiento Integral

1. Enunciado del ejercicio

Diseñe un plan que incluya: backups, reorganización de índices, actualización de estadísticas y verificación de integridad.

2. Script de la solución en T-SQL

```
CREATE PROCEDURE sp_MantenimientoIntegralGradosTitulos
AS
BEGIN
    -- 1. Respaldo completo
    BACKUP DATABASE GradosTitulos
    TO DISK = N'C:\Backups\GradosTitulos_Full.bak'
    WITH COMPRESSION, INIT;

    -- 2. Reorganización de índices con fragmentación entre 10% y 30%
    DECLARE @sql NVARCHAR(MAX) = N'';
    SELECT @sql += N'ALTER INDEX [' + i.name + N'] ON [' + s.name + N'].[' + t.name
    + N'] REORGANIZE;' + CHAR(13)
    FROM sys.dm_db_index_physical_stats(DB_ID('GradosTitulos'), NULL, NULL, NULL, 'LIMITED') ps
    JOIN sys.indexes i ON ps.object_id = i.object_id AND ps.index_id = i.index_id
    JOIN sys.tables t ON i.object_id = t.object_id
    JOIN sys.schemas s ON t.schema_id = s.schema_id
    WHERE ps.avg_fragmentation_in_percent BETWEEN 10 AND 30 AND i.name IS NOT NULL;
    EXEC sp_executesql @sql;

    -- 3. Actualización de estadísticas
    EXEC sp_updatestats;

    -- 4. Verificación de integridad
    DBCC CHECKDB('GradosTitulos') WITH NO_INFOMSGS;
END
```

3. Justificación técnica de la solución aplicada

Combinar respaldo, reorganización de índices (adecuada para fragmentación moderada, más económica que un REBUILD), actualización de estadísticas y verificación de integridad en una sola rutina garantiza tanto la disponibilidad como el rendimiento de GradosTitulos.

4. Buenas prácticas utilizadas en el proyecto

- Usar REORGANIZE para fragmentación de 10-30% y REBUILD para fragmentación mayor a 30%.
- Programar la ejecución en ventanas de bajo uso del sistema.
- Ejecutar CHECKDB regularmente para detectar corrupción de forma temprana.
- Mantener un registro de cada ejecución de mantenimiento.

Proyecto 4: Gestión Corporativa de Retención

1. Enunciado del ejercicio

Diseñe una política institucional de respaldo y retención para cinco años de operación.

2. Script de la solución en T-SQL

```
CREATE TABLE PoliticaRetencionBackups (
    TipoBackup VARCHAR(20),
    DiasRetencion INT
);

INSERT INTO PoliticaRetencionBackups VALUES
('Completo', 1825), -- 5 años
('Diferencial', 90),
('Log', 30);
GO

CREATE PROCEDURE sp_AplicarPoliticaCorporativa
AS
BEGIN
    DECLARE @FechaLimiteFull DATETIME = DATEADD(DAY, -1825, GETDATE());
    EXEC msdb.dbo.sp_delete_backuphistory @oldest_date = @FechaLimiteFull;
END
```

3. Justificación técnica de la solución aplicada

Una tabla de retención escalonada refleja distintas necesidades regulatorias/operativas según el tipo de respaldo: los completos se conservan por más tiempo por motivos de cumplimiento, mientras que los logs, al ser útiles solo a corto plazo, tienen una retención menor.

4. Buenas prácticas utilizadas en el proyecto

- Archivar los respaldos completos de 5 años en almacenamiento de bajo costo (cinta o nube fría).
- Documentar formalmente la política ante auditorías institucionales.
- Revisar la política de retención anualmente.
- Planificar adecuadamente el espacio en disco necesario para la retención de largo plazo.

Proyecto 5: Implementación de un Plan Corporativo de Mantenimiento

1. Enunciado del ejercicio

Diseñe un plan de mantenimiento que incluya: respaldo completo, reorganización de índices, actualización de estadísticas y validación de integridad.

2. Script de la solución en T-SQL

```
-- Reutiliza sp_MantenimientoIntegralGradosTítulos (Tema 5, Proyecto 3)
EXEC msdb.dbo.sp_add_job @job_name = N'Plan_Mantenimiento_Corporativo';

EXEC msdb.dbo.sp_add_jobstep
    @job_name = N'Plan_Mantenimiento_Corporativo',
    @step_name = N'Ejecutar Mantenimiento Integral',
    @subsystem = N'TSQL',
    @command = N'EXEC sp_MantenimientoIntegralGradosTítulos;',
    @database_name = N'GradosTítulos';

EXEC msdb.dbo.sp_add_schedule
    @schedule_name = N'Mensual_Corporativo',
    @freq_type = 16,
    @freq_interval = 1,
    @active_start_time = 010000;

EXEC msdb.dbo.sp_attach_schedule
    @job_name = N'Plan_Mantenimiento_Corporativo',
    @schedule_name = N'Mensual_Corporativo';

EXEC msdb.dbo.sp_add_jobserver
    @job_name = N'Plan_Mantenimiento_Corporativo';
```

3. Justificación técnica de la solución aplicada

Encapsular el mantenimiento integral (Tema 5, Proyecto 3) dentro de un Job de SQL Server Agent con programación mensual convierte una tarea manual en un proceso corporativo estandarizado, auditable y recurrente.

4. Buenas prácticas utilizadas en el proyecto

- Centralizar todos los planes corporativos en un servidor de administración.
- Configurar alertas de fallo para cada paso del Job.
- Revisar el historial de ejecución mensualmente.
- Documentar el plan como parte del manual de operaciones de TI.

Proyecto 6: Implementación de Política de Retención Automatizada

1. Enunciado del ejercicio

Diseñe una política institucional que elimine automáticamente respaldos con más de 180 días de antigüedad.

2. Script de la solución en T-SQL

```
CREATE PROCEDURE sp_RetencionAutomatizada180
AS
BEGIN
    DECLARE @FechaLimite DATETIME = DATEADD(DAY, -180, GETDATE());
    EXEC msdb.dbo.sp_delete_backuphistory @oldest_date = @FechaLimite;

    DECLARE @cmd NVARCHAR(500) =
        N'forfiles /p "C:\Backups" /d -180 /m *.bak /c "cmd /c del @path"';
    EXEC xp_cmdshell @cmd;
END
GO

EXEC msdb.dbo.sp_add_job @job_name = N'Job_RetencionAutomatizada';

EXEC msdb.dbo.sp_add_jobstep
    @job_name = N'Job_RetencionAutomatizada',
    @step_name = N'Eliminar backups mayores a 180 días',
    @subsystem = N'TSQL',
    @command = N'EXEC sp_RetencionAutomatizada180;';

EXEC msdb.dbo.sp_add_schedule
    @schedule_name = N'Mensual_Retencion',
    @freq_type = 16,
    @freq_interval = 1,
    @active_start_time = 020000;

EXEC msdb.dbo.sp_attach_schedule
    @job_name = N'Job_RetencionAutomatizada',
    @schedule_name = N'Mensual_Retencion';

EXEC msdb.dbo.sp_add_jobserver
    @job_name = N'Job_RetencionAutomatizada';
```

3. Justificación técnica de la solución aplicada

Automatizar la política de 180 días mediante un Job mensual elimina la dependencia de una intervención manual, asegurando que la política de retención se cumpla de forma consistente y auditable.

4. Buenas prácticas utilizadas en el proyecto

- Restringir y monitorear el uso de xp_cmdshell por razones de seguridad.
- Validar el resultado del Job y generar alertas ante fallos.
- Mantener respaldos de archivo fuera de esta rotación cuando exista requerimiento legal.
- Documentar la política y su automatización en el manual institucional.

TEMA 6. Restauración Punto en el Tiempo

Proyecto 1: Recuperación Antes de una Eliminación Accidental

1. Enunciado del ejercicio

Un operador eliminó registros de expedientes a las 10:15 a.m. Restaure la base exactamente a las 10:14 a.m.

2. Script de la solución en T-SQL

```
USE master;
ALTER DATABASE GradosTítulos SET SINGLE_USER WITH ROLLBACK IMMEDIATE;

RESTORE DATABASE GradosTítulos
FROM DISK = N'C:\Backups\GradosTítulos_Full.bak'
WITH NORECOVERY, REPLACE;

RESTORE DATABASE GradosTítulos
FROM DISK = N'C:\Backups\GradosTítulos_Diff.bak'
WITH NORECOVERY;

RESTORE LOG GradosTítulos
FROM DISK = N'C:\Backups\GradosTítulos_Log.trn'
WITH RECOVERY, STOPAT = '2026-07-03 10:14:00';

ALTER DATABASE GradosTítulos SET MULTI_USER;
```

3. Justificación técnica de la solución aplicada

La cláusula STOPAT en RESTORE LOG permite recuperar el log de transacciones hasta el instante indicado, excluyendo así la transacción de eliminación accidental ocurrida a las 10:15. Esto requiere que la base opere en modelo de recuperación FULL.

4. Buenas prácticas utilizadas en el proyecto

- Confirmar el modelo de recuperación FULL antes del incidente, como prerequisite indispensable.
- Identificar la hora exacta de la eliminación mediante logs de aplicación o auditoría.
- Restaurar primero a una base de prueba/copia para validar el resultado antes de aplicar en producción.
- Comunicar oportunamente la ventana de indisponibilidad a los usuarios.

Proyecto 2: Recuperación de Cambios Incorrectos

1. Enunciado del ejercicio

Un proceso masivo actualizó incorrectamente estados de trámites.

2. Script de la solución en T-SQL

```
ALTER DATABASE GradosTítulos SET SINGLE_USER WITH ROLLBACK IMMEDIATE;

RESTORE DATABASE GradosTítulos FROM DISK = N'C:\Backups\GradosTítulos_Full.bak'
WITH NORECOVERY, REPLACE;
RESTORE DATABASE GradosTítulos FROM DISK = N'C:\Backups\GradosTítulos_Diff.bak'
WITH NORECOVERY;
RESTORE LOG GradosTítulos FROM DISK = N'C:\Backups\GradosTítulos_Log.trn'
WITH RECOVERY, STOPAT = '2026-07-03 09:45:00'; -- hora previa al proceso masivo

ALTER DATABASE GradosTítulos SET MULTI_USER;
```

3. Justificación técnica de la solución aplicada

Se aplica el mismo mecanismo de STOPAT, pero la marca de tiempo debe determinarse con precisión mediante logs de errores, historial de Jobs o marcas de modificación en las tablas, para no perder transacciones legítimas posteriores al proceso erróneo.

4. Buenas prácticas utilizadas en el proyecto

- Cruzar el historial de Jobs de SQL Server Agent y logs de error para determinar la hora exacta de la falla.
- Restaurar primero a una base paralela para comparar registros afectados antes de aplicar a producción.

- Documentar el incidente y su causa raíz en el proceso masivo.
- Revisar y corregir el proceso batch antes de reintentarlo.

Proyecto 3: Recuperación Selectiva de Incidente Crítico

1. Enunciado del ejercicio

Un error en producción afectó las tablas Tramite y Evaluacion. Diseñe una recuperación punto en el tiempo minimizando la indisponibilidad.

2. Script de la solución en T-SQL

```
-- Restaurar una copia paralela para extraer los datos correctos sin afectar producción
RESTORE DATABASE GradosTítulos_Temp
FROM DISK = N'C:\Backups\GradosTítulos_Full.bak'
WITH MOVE 'GradosTítulos' TO 'D:\SQLData\GradosTítulos_Temp.mdf',
      MOVE 'GradosTítulos_log' TO 'D:\SQLLogs\GradosTítulos_Temp.ldf',
      NORECOVERY, REPLACE;

RESTORE LOG GradosTítulos_Temp
FROM DISK = N'C:\Backups\GradosTítulos_Log.trn'
WITH RECOVERY, STOPAT = '2026-07-03 08:30:00';

-- Reconciliar datos correctos hacia producción
INSERT INTO GradosTítulos.Tramite.Tramite
SELECT * FROM GradosTítulos_Temp.Tramite.Tramite t
WHERE NOT EXISTS (
    SELECT 1 FROM GradosTítulos.Tramite.Tramite p WHERE p.TramiteId = t.TramiteId
);
```

3. Justificación técnica de la solución aplicada

Restaurar a una base paralela (GradosTítulos_Temp) en lugar de sobrescribir producción minimiza la indisponibilidad, ya que el sistema principal permanece en línea mientras solo se reconcilian las filas afectadas desde la copia punto en el tiempo.

4. Buenas prácticas utilizadas en el proyecto

- Usar una convención de nombres clara para identificar bases temporales de recuperación.
- Eliminar la base temporal una vez concluida la reconciliación.
- Validar conteos y claves antes de fusionar los datos a producción.
- Ejecutar la operación de reconciliación en horario de bajo tráfico.

Proyecto 4: Simulación de Recuperación Forense Institucional

1. Enunciado del ejercicio

Se detectó una modificación maliciosa realizada durante una ventana específica de tiempo. Realice una recuperación forense que permita restaurar la base GradosTítulos exactamente al momento anterior al incidente y documentar todo el procedimiento.

2. Script de la solución en T-SQL

```
-- 1. Preservar evidencia: respaldar el log activo sin truncarlo
BACKUP LOG GradosTítulos TO DISK = N'C:\Backups\GradosTítulos_Log_Forense.trn'
WITH NO_TRUNCATE;

-- 2. Restaurar una copia forense en un servidor aislado
RESTORE DATABASE GradosTítulos_Forense
FROM DISK = N'C:\Backups\GradosTítulos_Full.bak'
WITH MOVE 'GradosTítulos' TO 'D:\Forense\GradosTítulos_Forense.mdf',
      MOVE 'GradosTítulos_log' TO 'D:\Forense\GradosTítulos_Forense.ldf',
      NORECOVERY, REPLACE;

RESTORE LOG GradosTítulos_Forense
FROM DISK = N'C:\Backups\GradosTítulos_Diff.bak'
WITH NORECOVERY;
```



```
RESTORE LOG GradosTitulos_Forense
FROM DISK = N'C:\Backups\GradosTitulos_Log_Forense.trn'
WITH RECOVERY, STOPAT = '2026-07-03 14:20:00'; -- momento previo al incidente

-- 3. Documentar hallazgos comparando contra produccion
SELECT * FROM GradosTitulos_Forense.Documento.Resolucion;
```

3. Justificación técnica de la solución aplicada

BACKUP LOG WITH NO_TRUNCATE preserva la cola del log incluso si el archivo de datos estuviera comprometido, siendo crucial como evidencia forense. Restaurar en una copia aislada (no en producción) evita alterar la evidencia; STOPAT aísla con precisión el estado previo al incidente para su análisis comparativo.

4. Buenas prácticas utilizadas en el proyecto

- Nunca restaurar copias forenses sobre el servidor de producción comprometido.
- Mantener documentación de cadena de custodia: quién ejecutó qué acción y cuándo.
- Usar transacciones marcadas (WITH MARK) de forma proactiva en operaciones críticas para facilitar futuros análisis forenses.
- Restringir el acceso a los archivos de respaldo forense a personal autorizado.

Proyecto 5: Recuperación Exacta de Expedientes Eliminados

1. Enunciado del ejercicio

A las 11:45 a.m. se eliminaron accidentalmente registros de expedientes de titulación. Recupere la base exactamente a las 11:44:59 a.m.

2. Script de la solución en T-SQL

```
ALTER DATABASE GradosTitulos SET SINGLE_USER WITH ROLLBACK IMMEDIATE;

RESTORE DATABASE GradosTitulos FROM DISK = N'C:\Backups\GradosTitulos_Full.bak'
WITH NORECOVERY, REPLACE;
RESTORE DATABASE GradosTitulos FROM DISK = N'C:\Backups\GradosTitulos_Diff.bak'
WITH NORECOVERY;
RESTORE LOG GradosTitulos FROM DISK = N'C:\Backups\GradosTitulos_Log.trn'
WITH RECOVERY, STOPAT = '2026-07-03 11:44:59';

ALTER DATABASE GradosTitulos SET MULTI_USER;
```

3. Justificación técnica de la solución aplicada

El uso de STOPAT con precisión al segundo garantiza excluir la transacción de eliminación (asumida confirmada a las 11:45:00), preservando la totalidad de los datos tal como existían un segundo antes.

4. Buenas prácticas utilizadas en el proyecto

- Verificar la sincronización de reloj entre la aplicación y el servidor SQL Server para asegurar exactitud del timestamp.
- Validar los datos restaurados mediante conteos de registros antes de liberar el acceso a usuarios.
- Documentar la línea de tiempo del incidente.
- Establecer controles de auditoría para prevenir eliminaciones accidentales futuras.

Proyecto 6: Recuperación Forense de una Modificación Maliciosa

1. Enunciado del ejercicio

Se detectó una modificación no autorizada en las tablas del esquema Documento relacionadas con resoluciones y diplomas. Realice una recuperación punto en el tiempo que permita restaurar el estado exacto de la base antes de la alteración.

2. Script de la solución en T-SQL

```
-- Preservar la cola del log antes de restaurar
BACKUP LOG GradosTitulos TO DISK = N'C:\Backups\GradosTitulos_Log_Incidente.trn'
```

```

WITH NO_TRUNCATE;

ALTER DATABASE GradosTítulos SET SINGLE_USER WITH ROLLBACK IMMEDIATE;

RESTORE DATABASE GradosTítulos FROM DISK = N'C:\Backups\GradosTítulos_Full.bak'
WITH NORECOVERY, REPLACE;
RESTORE DATABASE GradosTítulos FROM DISK = N'C:\Backups\GradosTítulos_Diff.bak'
WITH NORECOVERY;
RESTORE LOG GradosTítulos FROM DISK = N'C:\Backups\GradosTítulos_Log_Incidente.trn'
WITH RECOVERY, STOPAT = '2026-07-03 15:10:00'; -- hora previa a la alteracion detectada

ALTER DATABASE GradosTítulos SET MULTI_USER;

-- Auditoria posterior
DBCC CHECKDB('GradosTítulos') WITH NO_INFOMSGS;
SELECT * FROM Documento.Resolucion;
SELECT * FROM Documento.Diploma;

```

3. Justificación técnica de la solución aplicada

Capturar la cola del log con NO_TRUNCATE antes de restaurar garantiza no perder transacciones confirmadas; STOPAT restaura la base exactamente al último estado consistente antes de la modificación no autorizada, protegiendo la integridad de los registros académicos de grados y títulos.

4. Buenas prácticas utilizadas en el proyecto

- Habilitar SQL Server Audit o triggers sobre las tablas sensibles de Documento para detectar cambios no autorizados más rápido.
- Restringir los permisos de escritura sobre las tablas de resoluciones y diplomas a roles específicos.
- Revisar los logs de acceso e inicio de sesión para identificar la cuenta responsable.
- Reportar el incidente conforme a la política institucional de seguridad de la información.